



Attested Credit Protocol

A conversational agent for on-chain finance whose every consequential action is gated by Attestcoin proofs verified on the Creditcoin chain: proof-gated escrow and cross-chain swaps, attestation-aware automation, and payment certificates anyone can verify. No centralized oracle operator. No trust required.

ACP.ai Team · acpai-org

github.com/acpai-org/ACP.ai · acp-ai.vercel.app

1. What ACP.ai Is

ACP.ai is a conversational, multilingual agent for on-chain finance, built on Creditcoin and entered in the DeFi and AI tracks of BUIDL CTC 2026 Fall. One sentence captures the design: **Attestcoin is not a feature of ACP.ai - ACP.ai is an application of Attestcoin.** Every cross-chain fact the agent reasons about, every fund release it executes, and every proof a user exports is gated by on-chain verification on Creditcoin - no oracle operator, no “trust us,” no screenshots.

11 Attestcoin-native agent tools	0x0FD2 Block Prover Precompile	110 automated tests	4 languages, fully translated
--	--	-------------------------------	---

The submission pairs a closed tool-calling loop - 28 vetted tools, 11 of them Attestcoin-specific - with two families of Attestcoin Smart Contracts (ASCs) written for this project (Table 1), and bounds the model's discretion with eleven built-in, user-extensible skills whose per-skill tool allowlists are validated against the same registry. In both contract families verification and release are atomic: funds never move without the precompile accepting the proof, and “verified but not released” can never get stuck. The beta runs live on CC3 testnet, the code is open source, and a 110-case test suite covers the agent loop, fund-safety invariants, contract compilation to real bytecode, recurring-payment timing, skills seeding and versioning, the Attestcoin pipeline, and a tool-coverage suite asserting that every registry tool parses its sample, has a dispatch case, and has trace labels in all four locales.

The problem is old and simple: a transaction hash is not proof - it is a pointer. When a counterparty claims “I paid,” the usual answers are “trust the operator” or “go verify it yourself,” and neither scales to escrow, conditional settlement, or machine-to-machine payments. Centralized oracles concentrate exactly that trust in one falsifiable party [1]. ACP.ai closes the proof, usability, and autonomy gaps the same way - by refusing to be the authority: the agent is the interface, the wallet signs, the chain verifies.

2. How Attestcoin Verifies

The Attestcoin Protocol is a cross-chain execution layer built into the Creditcoin runtime: readability reads data from another blockchain into Creditcoin contracts, and writability sends messages that trigger execution on a destination chain [1]. Independent attestors continuously assert their view of source-chain state, and Creditcoin commits an attestation only after verifying their consensus, delivered as an aggregated BLS signature [2]. From committed attestations, anyone can construct the two proofs of any transaction: a Merkle proof of inclusion in a specific block, and a continuity proof that the block belongs to the real, finalized source chain.

Both proofs are checked by the Block Prover Precompile at 0x0FD2 - native Rust runtime code rather than EVM bytecode, so verification completes synchronously in one Creditcoin block, about 15 seconds [1][2]. It exposes `verify()` for read-only checks, `verifyAndEmit()` for on-chain evidence, and batch verification of up to 10 queries sharing one continuity proof; `ChainInfo` (0x0FD3) reports live coverage - currently Ethereum Sepolia and Ethereum mainnet. One documented subtlety is implemented deliberately: the precompile proves inclusion, not execution success - a reverted transaction is still “included” - so the docs require an ASC to check receipt status [2]; ACP.ai enforces that gate in both

ASC families, the decoder, and certificate verification. Figure 1 shows the end-to-end flow - the documentation's own dApp design pattern [3] - with the agent orchestrating what a human would otherwise do manually.

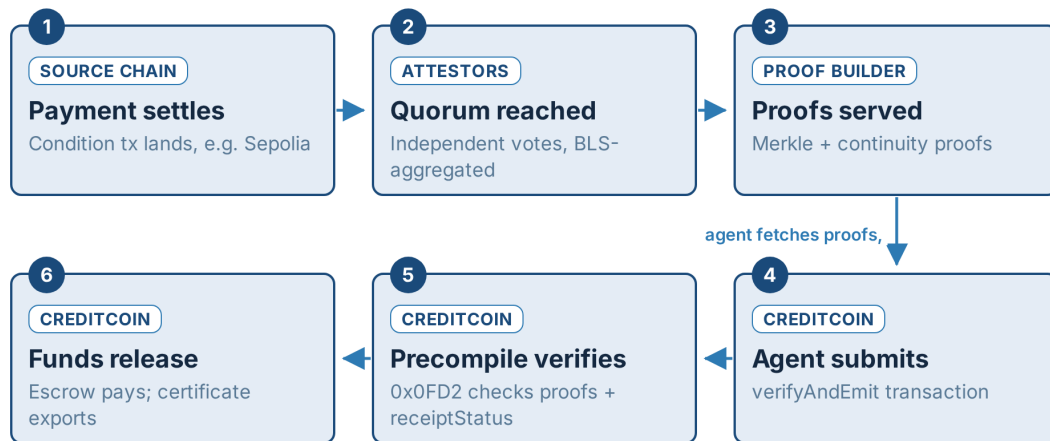


Figure 1: The end-to-end attestation flow (payment, proof, certificate, release)

3. Attestcoin as the Core Feature

The judging criterion is Attestcoin used “for a core feature, in a meaningful way,” so the team treated the protocol's documentation surface as a checklist: Table 1 lists the six integrations that make Attestcoin the core of the product, and Table 2 maps every documented primitive to the code that consumes it. The official @gluwa/usc-sdk (v0.18.0) underlies all ten modules of src/lib/attestcoin/ - now including a live network-status reader that reports attested heights and attestation lag straight from the ChainInfo precompile, and a CC3 deep-link module that turns every Attestcoin record into a viewable Blockscout URL. The vendored INativeQueryVerifier interface stays byte-identical with the precompile's ABI [7].

Integration	What it does	Where it lives
Proof-gated fund releases	Escrow pays only when a Merkle + continuity proof passes 0x0FD2 in the same tx - atomic verify-and-release.	ConditionalRelease.sol (ASC)
Proof-gated cross-chain swaps	ETH locks on Sepolia; tCTC releases only against a verified proof of the lock - rate fixed at lock time.	CrossChainSwapSource / Destination
Attestation-aware agent tools	11 tools: check and wait for attestation, decode proven txs, verify read-only, estimate cost, submit on-chain, report network status and lag.	src/lib/agent/tool-registry.ts
Attestation-driven automation	User rules fire on attestation_ready, balance thresholds, and schedules - Creditcoin-verified state, not explorer polls.	automation_rules / poller.ts
Portable certificates	Any attested payment exports as JSON + QR; anyone re-runs five live checks without ACP.ai.	certificate-verifier API
Skill-governed autonomy	11 built-in, user-extensible skills; per-skill tool allowlists validated against the closed registry.	src/lib/skills.ts, /api/skills

Table 1: The six core Attestcoin integrations

Attestcoin primitive	Where ACP.ai uses it
Official SDK - @gluwa/usc-sdk (v0.18.0)	All ten modules of src/lib/attestcoin/ - proof.ts, batch.ts, verify.ts, submit.ts, chains.ts, decode.ts, poller.ts, status.ts, cc3-links.ts, config.ts
Proof Builder getProof / getBatchProof	proof.ts, batch.ts - per-tx and batch fetches, shared continuity, up to 10 txs per call
Block Prover 0x0FD2 - read path	verify.ts - verifySingle / verifyBatch as read-only eth_call; used by the poller, routes, and certificates
Block Prover 0x0FD2 - write path	submit.ts - verifyAndEmitSingle / Batch as signed transactions; TransactionVerified parsed from receipts
ChainInfo precompile 0x0FD3	chains.ts - live getSupportedChains(); chainKey map derived at runtime, never hardcoded. status.ts - attested heights and live attestation lag per source chain
EvmV1Decoder contract	decode.ts - decodes from, to, value, receipt, and logs from the proven transaction bytes
ASC pattern (docs best practice)	ConditionalRelease.sol, CrossChainSwapDestination.sol on the vendored INativeQueryVerifier; minimal Locked-event source [3]
Off-chain readability worker	The 60-second poller: waits for attestation, fetches proofs, chunks to limits, notifies exactly once
Batch caps and continuity merge	chunkByProtocolLimits enforces the 10-proof / 1000-block caps; tryMergeProofs merges contiguous ranges, falls back to per-tx
Receipt-status gate (docs: ASCs MUST check)	EvmV1Decoder in both ASCs, decode.ts, and certificate check five of five
CC3 explorer deep links	cc3-links.ts - Blockscout transaction links for every Attestcoin record, derived from one chain registry

Table 2: Protocol surface coverage - every documented primitive, and where ACP.ai uses it

4. Proof-Gated Architecture

All agent behavior runs through one tool-calling loop over a closed registry (Figure 2). The model proposes tool calls; only names present in the registry survive - unknown names are rejected and logged, never interpreted. Reads run in-process on the server; fund moves, deployments, and swaps dispatch to the browser wallet; results feed back to the model for at most eight bounded rounds, with hard timeouts - ten minutes per tool result, five minutes per confirmation - and a browser disconnect aborts the run, marks pending steps interrupted, and lets on-chain transactions finish independently while the log catches up from the receipt. The model supplies intent only: the browser converts validated intents into known transaction shapes and the wallet signature is the confirmation, so raw calldata from the model is structurally impossible. Deployments show a source-plus-plain-English confirmation card, and mainnet requires an opt-in flag that ships off by default. On retry, completed steps are mapped to the model's tool role so only the failed step re-runs - the agent cannot double-pay by repeating a turn.

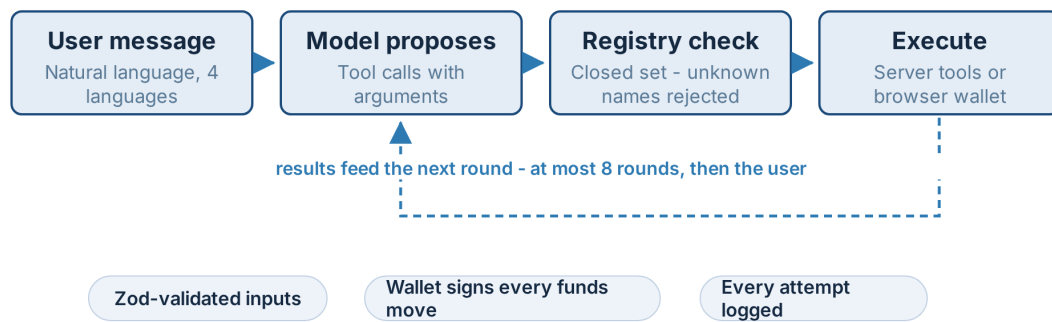


Figure 2: The agent loop - closed registry, two executors, bounded rounds

The deployment view keeps custody where it belongs (Figure 3). The browser holds the UI, the wallet, and the model-provider key, which never reaches the server; the Node server runs the agent loop, the tool registry, an in-process Solidity 0.8.36 compiler, the attestation poller, and 34 API routes over a local SQLite store - Node's built-in node:sqlite behind drizzle-orm, eight tables, migrated at runtime - that logs every tool attempt with its proof references. A single chain registry is the source of truth for nine wallet chains (Creditcoin testnet and mainnet, Ethereum and Sepolia, Base, Arbitrum One, OP Mainnet, Polygon PoS, and BNB Smart Chain) with automatic chain-switching through Reown AppKit. Everything else - model endpoint, EVM chains, Proof Builder, Creditcoin - is external, and the application sends no telemetry. English, Korean, Chinese, and Japanese are fully translated - 1,137 keys per locale with compile-checked parity enforced by the type system and a test-time sweep.

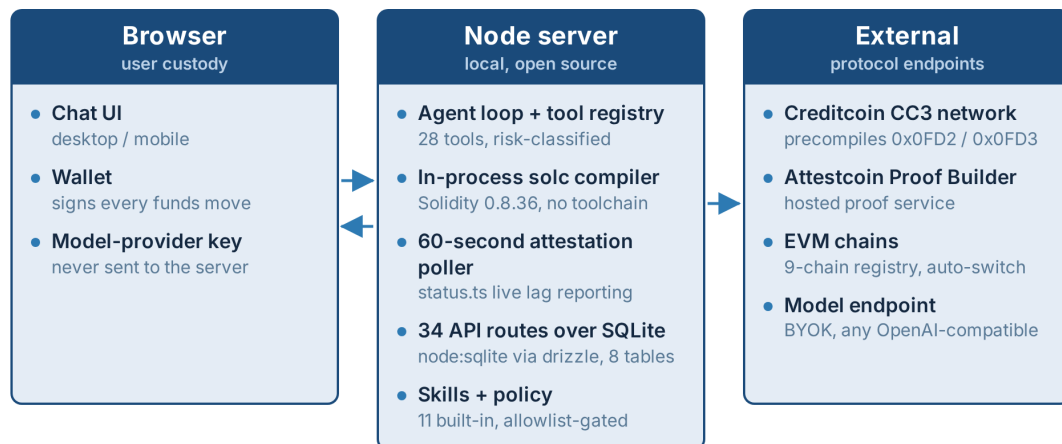


Figure 3: Deployment architecture - browser, server, and external endpoints

Skills: bounded autonomy the user controls

Skills are named instruction packets the user can enable, disable, and author - a layer of policy that sits between the model and the tool registry. Each skill carries an optional tool allowlist; the list is validated against the closed registry on every write, so a skill can only ever narrow what the agent may call, never widen it. The semantics are pinned deliberately: a missing allowlist means all tools, a stored list whose tools have all been removed from the registry serializes as an honest empty list rather than silently reverting to all tools, and a list of nothing but unknown names is rejected outright instead of being widened. The allowlist rules, serialization, and seeding logic are pinned by 20 unit cases in the skills-allowlist and skills-seed suites.

The library ships with eleven built-in skills seeded at first boot - from the exact-change payer that restates full addresses and amounts before every transfer, and the Attestcoin flow operator that narrates lock, attestation-wait, and proof-backed release with transaction hashes, to the testnet guardian that names mainnet spends explicitly and the attestation sentinel that refuses to call a proof verified until the precompile says so. Builtins are versioned: each spec carries a content hash, a changed skill refreshes its instructions and allowlist on boot, and a refresh never re-enables a skill the user deliberately turned off. Users extend the same library through the `/api/skills` routes with identical validation.

Skills matter to the security narrative as well: enabled skills constrain the patterns the model draws on when it generates a custom contract, which narrows - though it does not eliminate - the hallucination risk of LLM-authored Solidity. Together with the mandatory confirmation card, they form the human-control half of the autonomy story: the registry bounds what the agent can call, the allowlist bounds what a skill permits, and the wallet bounds what actually moves.

5. Correctness and Security Invariants

The Attestcoin documentation is explicit about the details that bite if you skip them - receipt status, batch limits, chainKey identity, proof staleness, replay. ACP.ai implements each defense deliberately, pinned by 25 unit cases in the attestcoin-pipeline and attestcoin-gaps suites (Table 3).

Hazard	Defense in ACP.ai
Precompile proves inclusion, not execution - a reverted tx is still “included”	receiptStatus == 1 checked before any funds move - in both ASCs, decode.ts, and certificate check 5 of 5
Batch caps: at most 10 proofs spanning under 1000 blocks	chunkByProtocolLimits sorts by height and cuts chunks that respect both caps; unit-tested
Shared continuity proofs only sometimes available	tryMergeProofs merges only contiguous coverage; a gap degrades to per-tx rather than risking a revert after gas
chainKey is not chainId - and differs per environment (Sepolia is 1 on testnet, Ethereum is 1 on mainnet)	The chainKey map is derived live from 0x0FD3, so a wrong-chain query is structurally impossible
Continuity proofs grow 10-100x costlier to verify as the chain advances	Estimated cost and staleness gap surfaced before submission, not as a reverted transaction
Replay: resubmitted proof, refunded lock re-paid, forged Locked event	One-shot release state machine, per-lock spentLocks, expiry gating before refund, trusted-source pinning

Table 3: Protocol hazards and ACP.ai's defenses

Verification cost is modeled, not discovered. Following the documentation's cost model, the estimated fee is 2.3×10^{-5} CTC plus 2.9×10^{-7} CTC per continuity root, with a 10-100x stale-proof penalty; ACP.ai surfaces the estimate and the staleness gap before the user signs [1]. Gas for wallet actions comes from live estimateGas on the exact transaction shape, never a constant - and the product shows no USD valuations anywhere, so it can never display a stale price.

The security model reduces to invariants the application will not violate. Nothing moves without a wallet signature; the toolset is closed and Zod-validated; proof submission runs through a dedicated, optional server signer key that neither the agent nor the browser can read or choose - and the submit tool refuses with instructions when the key is unset, rather than failing silently; every tool attempt lands in the action log with its transaction hash and Merkle root; and the model-provider key lives in browser storage, never on the server. One residual risk is stated plainly: LLM-authored contracts carry hallucination risk that vetted templates do not. Enabled skills constrain the generation patterns and the confirmation card is the final human checkpoint, but automated static analysis - reentrancy, access control, overflow checks - is not yet integrated; it remains first on the roadmap, and until it ships, custom deployment is a user-owned decision with risks displayed at confirmation time.

6. Scope and Competition

The current scope is stated exactly. Attestation coverage today is Sepolia and Ethereum mainnet, queried live from 0x0FD3 - payments on other chains settle normally but do not attest, and the pipeline extends automatically as the attester network grows. Attestcoin's writability path - an Outbox, attester validation, and permissionless relayers delivering signed messages to a destination Inbox - is not yet live on testnet while third-party audits are in progress, so conditional releases currently resolve on Creditcoin; when writability ships, the agent already speaks in proof-shaped intents, making the upgrade a contract-layer change, not a redesign. The demo runs on CC3 testnet against live protocol endpoints.

Approach	Verification authority	ACP.ai's position
Centralized oracle / bridge	A single operator's honesty; compromise can steal funds or falsify data	Eliminated by design - no operator between agent and facts
Oracle DON (Chainlink CCIP)	Staked committee plus risk-management networks; mature, widely integrated	Strong, but trust rests on committee economics, not chain runtime
Guardian quorum (Wormhole)	Signature threshold across identifiable guardian nodes	Still a quorum of operators, not the chain itself
General crypto AI agents	Explorer APIs and unverified model knowledge; no proof layer	Facts are precompile-verified before the agent reasons
Attestcoin Protocol (this project)	Native runtime precompile 0x0FD2 on Creditcoin; attester consensus on-chain	Verification is the chain's own code - synchronous, permissionless

Table 4: Who verifies the cross-chain facts?

The meaningful comparison class is not “AI chatbots” but systems that move value on cross-chain facts - and every one must answer: who verifies the facts? Chainlink CCIP and Wormhole are battle-tested at massive scale with economically staked committees; nothing here claims otherwise. The distinction is architectural: there, verification is a signature quorum of identifiable operators between the chains; in

Attestcoin it is native runtime code of a layer-1 - synchronous, permissionless, with no operator whose compromise would forge a proof [1][2]. ACP.ai's contribution is the demonstration that a consumer-grade agent and production-grade DeFi flows can run entirely on that native verification (Table 4).

7. Roadmap and Deliverables

The roadmap sequences engineering against the protocol's own maturity - the disclosed static-analysis gap first, audits before any mainnet value at risk, and the writability upgrade the moment it ships (Table 5). For the DeFi track, the submission demonstrates trustless cross-chain value transfer on Creditcoin: an escrow whose release condition is a proven foreign transaction, a swap whose legs lock and release against proofs rather than promises, and certificates anyone can verify. For the AI track, it is an agent whose autonomy is bounded by design - a closed, risk-classified registry, skill-scoped allowlists, human-gated deployments, and precompile-verified inputs to every cross-chain decision. The tracks meet in one sentence: the agent is the interface, Attestcoin is the authority - and everything is inspectable: open source, a live beta on CC3 testnet, and a one-command test suite.

Horizon	Milestone	Why it matters
Near term	Automated static analysis (reentrancy, access control, overflow) before every custom deployment; standalone web certificate verifier; external audit of the ASC suite	Closes the disclosed residual risk; third parties verify payments with zero install; audits precede mainnet value
Protocol-driven	Writability upgrade: destination-chain resolution via Outbox, attester quorum, relayer, Inbox - the agent layer is already built for it	Proof-gated escrow becomes bidirectional settlement
Program	Creditcoin Ecosystem Investment Program (CEIP)	Investment, engineering and product advisory, and the partner network to fast-track ACP.ai from beta to launch

Table 5: Roadmap milestones

Resource	Where
Source code (open source)	github.com/acpai-org/ACP.ai
Live beta (CC3 testnet)	acp-ai.vercel.app
Test suite (110 cases, 10 files)	<code>npm run test</code> in the repository
Project X (Twitter)	x.com/acp_ai
Local setup	Node.js 24 or later; <code>npm install</code> ; <code>cp .env.example .env</code> ; set <code>ATTESTCOIN_NETWORK</code> (testnet or mainnet) and the optional <code>CREDITCOIN_SIGNER_KEY</code> ; <code>npm run build</code> ; <code>npm run start</code>

Table 6: Deliverables and resources

ACP.ai set out to close the proof gap, the usability gap, and the autonomy gap, and it closes all three the same way: by refusing to be the authority. The application makes attested cross-chain finance conversational; the chain makes it true.

References

- [1] Attestcoin Protocol. (2026). Attestcoin Protocol - Overview. Attestcoin documentation.
<https://docs.attestcoin.org/>
- [2] Attestcoin Protocol. (2026). Architecture: Attestors, Validators, Block Prover Precompile, Message Relayers.
<https://docs.attestcoin.org/attestcoin-protocol/architecture>
- [3] Creditcoin. (2026). Source Chain Smart Contracts - dApp Builder Infrastructure.
<https://docs.creditcoin.org/attestcoin-protocol/dapp-builder-infrastructure/source-chain-smart-contracts>
- [4] Creditcoin. (2026). Creditcoin: The Layer 1 Built for Cross-Chain Builders. <https://creditcoin.org/>
- [5] ACP.ai Team. (2026). ACP.ai repository. GitHub. <https://github.com/acpai-org/ACP.ai>
- [6] ACP.ai Team. (2026). ACP.ai live beta. <https://acp-ai.vercel.app>
- [7] Gluwa. (2026). @gluwa/usc-sdk (v0.18.0) [Software library]. npm.
<https://www.npmjs.com/package/@gluwa/usc-sdk>